Master Admin ⇄ OneCity apps — handshake

How the Master Admin (the founder's control panel) and the customer, rider and merchant apps share one Supabase database, and how an edit in the admin reaches a customer's phone.

Rule: the database is the contract. The admin writes rows; the apps read them through our own API (`frontend/app/api/*`). No app talks to another app directly. Production (`ywylhzcxuoeilygzdonx`) changes are additive only, with a backup first.

---

1. How an admin edit reaches the apps



- **Catalogue, prices, banners, coupons, flags:** the API reads the database on each request (short cache, ≤60 s). Edits appear on the next screen open. Realtime makes it instant for open screens.
- **Orders:** status changes (merchant accepts, rider picks up) are written to `ol_orders` and pushed over Realtime; the customer's tracking screen, widget and Live Activity update from that.
- **UI/code changes** (new screen, layout) are not admin edits — they ship by OTA (`docs/OTA_UPDATES.md`) or a store build.

2. Tables

Already exist (0021–0023 applied to production; 0024 applied locally, production PENDING)

Table	Admin edits	Used by
`ol_categories`, `ol_products`, `ol_brands`, `ol_stores`, `ol_themed_stores`	Catalogue CRUD, stock, price/MRP, images	customer
`ol_media_assets`	Image/video library	all
`ol_banners`, `ol_discover_chips`, `ol_reels`, `ol_events`	Home/discover content	customer
`ol_coupons`	Discount codes	customer, pricing
`ol_service_areas`	Delivery zones per city	customer, rider
`ol_orders`, `ol_order_items`	Status, notes (read mostly)	all three
`ol_refunds` (+ `ol_process_refund()`)	Issue refunds	admin, customer
`ol_booking_enquiries`, `ol_support_requests`	Respond / close	admin
`ol_merchant_channels`, `ol_partner_devices`	Link restaurants, rider phones	merchant, rider

To create (next migration, 0025 — additive)

Table	Purpose	Key columns

`ol_feature_flags`	Switch features per city without a deploy (replaces env logic in `server/features.ts`)	`key`, `city_id`, `enabled`, `rollout_pct`, `note`, `updated_by`, `updated_at`
`ol_pricing_rules`	Everything in `PRICING` (`server/pricing.ts`): delivery fee, free-delivery threshold, handling/platform fee, food GST, delivery GST	`city_id`, `key`, `value`, `valid_from`, `valid_to`
`ol_tax_rates`	GST by category or HSN (today sample rates in code, to confirm with CA)	`category_id` / `hsn`, `rate_pct`
`ol_discounts`	Automatic discounts (store-wide %, category sale, BOGO) beyond codes	`scope`, `scope_id`, `type`, `value`, `min_cart`, `starts_at`, `ends_at`, `max_uses`, `active`
`ol_import_jobs`	Bulk upload tracking	`id`, `kind` (products/prices/stock/images), `file_path`, `status`, `rows_total`, `rows_ok`, `errors jsonb`, `created_by`
`ol_audit_log`	Every admin write	`actor`, `table`, `row_id`, `action`, `before jsonb`, `after jsonb`, `at`
`ol_admin_users`	Who may use the admin, and at what level	`user_id`, `role` (owner/ops/catalog/support/finance), `city_ids`
`ol_notifications`	Push campaigns & templates	`title`, `body`, `audience`, `send_at`, `status`
`ol_customer_profiles`, `ol_addresses`	Move profile/addresses from the phone to the DB	per customer
`ol_coins_ledger`	OneLocal Coins (see `ONELOCAL_COINS_PLAN.md`)	`customer`, `delta`, `reason`, `order_id`, `expires_at`

All new tables: `city_id`, `created_at`, `updated_at`, RLS on, writes only through the service role (admin API) — never from a phone.

3. Master Admin screens needed

1. **Dashboard** — today's orders, revenue, cancellations, live riders.
2. **Catalogue** — categories, products (price, MRP, stock, images, GST rate), brands, stores; bulk edit.
3. **Bulk upload** — CSV/XLSX template download → upload → preview with row errors → confirm → `ol_import_jobs` shows progress. Images by ZIP or URL column. Nothing is written until confirmed.
4. **Pricing & fees** — edit `ol_pricing_rules`; preview a sample bill before saving.
5. **Discounts & coupons** — create, schedule, cap uses, see redemptions.
6. **Feature flags** — on/off per city (AI search, maps, online payments, wallet, coins, Google sign-in).
7. **Content** — banners, discover chips, reels, themed stores; drag to reorder; schedule.
8. **Orders** — list/filter, detail, change status, assign rider, refund (calls `ol_process_refund`).
9. **Merchants & riders** — onboarding, devices, channels, payouts (later).
10. **Customers & support** — lookup, support requests, enquiries.
11. **Notifications** — compose push, audience, schedule.
12. **Delivery zones** — draw/edit `ol_service_areas`.
13. **Admins & audit** — roles, and the audit log viewer.

4. API contract

- Admin writes go through `/api/admin/*` routes, checked against `ol_admin_users` role, each writing `ol_audit_log`. (Not built yet.)

- Apps keep today's reads unchanged: `GET /api/features`, `POST /api/cart/quote`, catalogue routes, `/api/orders/*`. Swapping env/code values for table values happens behind these routes, so apps need no update.
- Money is always recomputed on the server (`quote()`); the admin changes rules, never totals on an existing order.

5. Order of work

1. Founder approves production migration 0024 (0021–0023 already applied; backup first).
2. Migration 0025 with the tables above; seed from current code values so nothing changes on day one.
3. Point `features()` and `PRICING` at the tables (with code values as fallback).
4. Build admin: flags → pricing → catalogue + bulk upload → orders/refunds → rest.
5. Realtime channel for instant refresh.

6. Decisions only the founder can make

- Handling fee, platform fee, GST on delivery — amounts (today 0).
- GST rates per category (CA to confirm).
- Who gets which admin role; is refund approval one step or two?
- Will the Master Admin be a separate web app or a section of the existing OS?

---

7. Entire app config through the Master Admin — `ol_app_config`

Anything a customer sees that is not code must be a row the admin can edit. One table holds the app-wide settings as typed keys per city (and per app: `customer` / `rider` / `merchant`):

Key	Type	Controls today (hard-coded)
`home.sections`	ordered list	Order/visibility of home rails (hero spotlight, departments, stores, picks, budget, new brands)
`home.spotlight`	object	Hero card (brand, headline, image, CTA)
`departments`	list	Tabs Food / Groceries / Shops / Pharmacy & Beauty / Book It — name, icon, colour, on/off
`theme.accent`	colour	Neon lime `#B3F400` and department palettes
`festival`	object	Festival theme + mascot (on/off, dates, artwork)
`delivery.eta_text`	text	"10 min" promise text
`support.phone`, `support.whatsapp`, `support.email`	text	Help screen contacts
`legal.terms_url`, `legal.privacy_url`	url	Account → About
`app.min_version`	semver	Force-update prompt below this build
`app.maintenance`	object	Kill switch: `{on, message}` — app shows a friendly closed screen
`checkout.cod_limit`	₹	Max cash-on-delivery order
`checkout.min_order`	₹	Minimum cart value
`copy.*`	text	Headlines, search placeholder ("A little search. A lovely find."), empty states

```
create table public.ol_app_config (
  app text not null default 'customer' check (app in ('customer','rider','merchant')),
  city_id text not null default 'latur' references public.ol_service_areas(city_id),
  key text not null, value jsonb not null,
  updated_by uuid, updated_at timestamptz not null default now(),
  primary key (app, city_id, key)
);
```

The app reads it once at launch from `GET /api/config` (to build; returns all keys merged with code defaults, cached 60 s) together with `GET /api/features`. A missing key always falls back to today's built-in value, so an empty table changes nothing.

8. Live API reference (built and running today)

Base URL: the app's own origin (`http://localhost:8081` in development; the Vercel domain in production). All responses are JSON. Errors: `{ "detail": "message" }` with an HTTP status.

Headers

- `x-customer-key` — random private key per device (guest identity). Sent by the app automatically.
- `authorization: Bearer <supabase access token>` — when the customer is signed in.
- Partner routes: `authorization: Bearer <token>` of a merchant/rider account linked in `ol_partner_devices` / `ol_merchant_channels`.
- Common query: `city` (default `latur`).

Customer app

Method	Route	Purpose	Reads
GET	`/api/health`	Liveness check	—
GET	`/api/features`	Which features are live vs "coming soon"	env → `ol_feature_flags` (planned)
GET	`/api/catalog`	Whole catalogue in one call (categories, stores, products, brands)	`ol_categories`, `ol_stores`, `ol_products`, `ol_brands`
GET	`/api/categories`	Category list	`ol_categories`
GET	`/api/products?q=&kind=&tab=`	Product list / filter	`ol_products`
GET	`/api/products/:id`	One product	`ol_products`
GET	`/api/products/:id/story`	Product page: gallery, video, store, brand	`ol_products`, `ol_stores`, `ol_reels`, `ol_media_assets`
GET	`/api/products/buy-again`	"Buy again" rail (sample for now)	—
GET	`/api/stores`	Store list	`ol_stores`
GET	`/api/stores/:id`	Store profile + media carousel	`ol_stores`
GET	`/api/stores/:id/products`	Store's products	`ol_products`
GET	`/api/banners`	Home banners	`ol_banners`
GET	`/api/discover`	Discover chips + content	`ol_discover_chips`
GET	`/api/reels`	Reels feed	`ol_reels`
GET	`/api/events`	Book It events	`ol_events`
GET	`/api/media/:id`	Image/video by id (redirect/stream)	`ol_media_assets`
GET	`/api/coupons`	Coupons shown in cart	`ol_coupons`

POST	`/api/coupons/apply`	Check a code against a subtotal	`ol_coupons`
POST	`/api/cart/quote`	Server bill: items, GST (CGST/SGST), delivery, coupon, smart top-ups	`PRICING` → `ol_pricing_rules` (planned)
GET	`/api/orders`	My orders	`ol_orders` , `ol_order_items`
POST	`/api/orders`	Place order (re-quotes; refuses mismatched total; idempotent)	writes `ol_orders` , `ol_order_items`
GET	`/api/orders/:id`	Order detail / tracking (404 if not mine)	`ol_orders`
GET	`/api/serviceability?lat=&lng=`	Do we deliver here?	`ol_service_areas`
GET	`/api/places/autocomplete?q=&session=`	Address search (Google, server-side key)	Google Places
GET	`/api/places/:id`	Place details	Google Places
GET	`/api/geocode/reverse?lat=&lng=`	Pin → address	Google Geocoding
POST	`/api/booking-enquiries`	Book It enquiry	writes `ol_booking_enquiries`
POST	`/api/support-requests`	Help request	writes `ol_support_requests`
GET/POST	`/api/search/*` , `/api/assistant/*`	AI search / assistant — returns "coming soon" until AI key	—

Merchant / rider (partner) apps

Method	Route	Purpose
GET	`/api/partner/me`	Who am I (merchant/rider, outlet)
PATCH	`/api/partner/outlet`	Open/close outlet, prep time
PUT	`/api/partner/devices`	Register phone for push/ringing
GET	`/api/partner/items`	Menu items
PATCH	`/api/partner/items/:id`	In stock / out of stock, price
GET	`/api/partner/orders`	Incoming + active orders
GET	`/api/partner/orders/:id`	Order detail
POST	`/api/partner/orders/:id/accept` | `reject` | `ready` | `out` | `delivered`	Order status steps
POST	`/api/partner/test-order`	Fire a test order to ring the device

9. Master Admin API (to build) — skeletons

All under `/api/admin/*` , `authorization: Bearer <token>` of a user in `ol_admin_users` . Every write returns the saved row and appends `ol_audit_log` . Validation errors → `422 { detail, fields }` .

Create a new store

`POST /api/admin/stores`

```
{
  "id": "latur-bakes",
  "name": "Latur Bakes",
  "tagline": "Fresh from the oven, every morning",
  "department": "food",
  "city_id": "latur",
  "delivery_time": "20 min",
  "tags": ["Bakery", "Cakes"],
  "logo": "media://latur-bakes-logo",
  "media": [
    { "type": "video", "url": "media://latur-bakes-reel", "thumbnail": "media://latur-bakes-thumb" },
    { "type": "image", "url": "media://latur-bakes-counter" }
  ],
  "organization_id": null,
  "is_demo": false,
  "sort_order": 10
}
```

→ `201` store row. Then `PATCH /api/admin/stores/:id` (any field), `DELETE` = set `active=false` (never hard delete; orders reference stores). Media first via `POST /api/admin/media` (returns a signed upload URL; the file goes straight to Supabase Storage).

Create a coupon

`POST /api/admin/coupons`

```
{ "code": "DIWALI100", "label": "₹100 off", "kind": "flat", "value": 100, "min_order": 499,
  "description": "Diwali offer, once per customer", "active": true, "city_id": "latur",
  "starts_at": "2026-10-18T00:00:00+05:30", "ends_at": "2026-10-24T23:59:59+05:30",
  "max_uses": 1000, "max_uses_per_customer": 1 }
```

(`starts_at` , `ends_at` , `max_uses*` are new columns in 0025.) Turn off: `PATCH /api/admin/coupons/DIWALI100 {"active": false}` . Redemptions: `GET /api/admin/coupons/DIWALI100/redemptions` .

Adjust a discount (automatic, no code)

`POST /api/admin/discounts` / `PATCH /api/admin/discounts/:id`

```
{ "scope": "category", "scope_id": "c9", "type": "pct", "value": 15, "min_cart": 0,
  "starts_at": "2026-10-01T00:00:00+05:30", "ends_at": null, "active": true, "city_id": "latur" }
```

`scope` : `all` | `store` | `category` | `product` | `brand` . `quote()` applies the best single discount per line, then the coupon; the bill shows it as its own line.

Pricing & fees

`PATCH /api/admin/pricing` `{ "city_id": "latur", "deliveryFee": 15, "freeDeliveryFrom": 199, "handlingFee": 0, "platformFee": 0, "foodGstPct": 5, "deliveryGstPct": 0 }` → response includes a sample bill so the admin sees the effect before customers do.

Feature flags

`PUT /api/admin/flags/:key` `{ "city_id": "latur", "enabled": true, "rollout_pct": 100 }`

App config

`GET /api/admin/config?app=customer&city=latur` → all keys with defaults `PUT /api/admin/config/:key` `{ "app": "customer", "city_id": "latur", "value": … }`

Catalogue & bulk upload

- `POST/PATCH /api/admin/products`, `/categories`, `/brands`, `/banners`, `/reels`, `/events`
- `POST /api/admin/imports { "kind": "products", "file": "storage path" }` → job id; `GET /api/admin/imports/:id` → progress + row errors; `POST /api/admin/imports/:id/commit` to apply.

Orders & refunds

- `GET /api/admin/orders?status=&from=&to=` · `PATCH /api/admin/orders/:id { status, note }`
- `POST /api/admin/orders/:id/refund { amount, reason }` → calls `ol_process_refund()`.

10. Admin UI skeleton



Each editor has the same three parts: **form → live preview (the real app screen, rendered from the unsaved values) → Save & publish**, and every list has a History tab from `ol_audit_log`. Coupons: list with uses/limit bar, `[+ New coupon]` drawer (code, flat/%, value, min order, dates, limits). Discounts: scope picker (all/store/category/product/brand) + value + dates. App config: grouped key editor (Home, Theme, Checkout, Support, App) with "reset to default" per key.